



Secureworks Taegis XDR Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-20

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	2
Verifying the setup.....	2
Reference documentation	3

Overview

The Taegis XDR connector authenticates to the Secureworks Taegis GraphQL APIs as a machine client using OAuth2 client credentials (a `client_id` and `client_secret`). A Taegis administrator creates an API client in the tenant, copies the generated credential once, and notes which regional API host the tenant lives in. These steps are performed by a customer-side Taegis administrator with permission to manage API clients — not by Crogl.

NOTE

Some steps below are marked **(TBD — verify against the live console)**. They are derived from the public Taegis API documentation and have not been confirmed first-hand against a tenant. Confirm them during connector field-testing and update this guide.

What kind of account to create

Taegis uses an **API client** (a machine-to-machine credential), not a named user login. In the Taegis console, an administrator creates a client, which yields a `client_id` and a `client_secret` used for the OAuth2 client-credentials grant.

The documented programmatic path creates a client by POSTing to the tenant's `clients` endpoint with an existing user access token, for example:

```
curl -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  https://<region_host>/clients/v1/clients \
  -d '{"name": "crogl-xdr-connector"}'
```

The console UI equivalent (Tenant Settings → API Access, or similar) is **(TBD — verify the exact menu path against the live console)**. Give the client a recognizable name such as `crogl-xdr-connector`.

Permissions to grant

The API client must be granted read access to the surfaces the connector uses: **alerts**, **investigations**, and **endpoint assets**. In Taegis these correspond to analyst-tier read roles/scopes on those services.

- The connector performs **read-only** GraphQL queries; it never needs write, remediation, or endpoint-isolation permissions. Grant the least-privilege read role available.
- The exact role/scope names to assign the client are **(TBD — verify against the live console)**; assign the tenant's standard read-only analyst role if a finer-grained per-service scope is not offered.

Generating the credential

When the client is created, Taegis returns the `client_secret` **once**. Copy it immediately and store it securely; it cannot be retrieved again — if lost, the client must be rotated (a new secret generated). The `client_id` remains visible on the client afterward.

Access tokens minted from these credentials are valid for approximately 10 hours (`expires_in: 36000`); Crogl re-mints them automatically, so no manual token refresh is required.

Mapping values into Crogl

One row per `community/taegis/connector.json` credential-schema field:

Crogl field	Where it came from	Notes
region_host	The tenant’s regional API host	No scheme, no path — just the host. US1 = api.ctpx.secureworks.com ; other regions in the table below. This host is used for both API calls and the token endpoint.
client_id	The created API client’s ID	Visible on the client after creation.
client_secret	The secret shown once at client creation	Copy at creation time; not retrievable later. Stored encrypted by Crogl.

Regional API hosts:

Region	region_host
US1	api.ctpx.secureworks.com
US2	api.delta.taegis.secureworks.com
US3	api.foxtrot.taegis.secureworks.com
EU1	api.echo.taegis.secureworks.com
EU2	api.golf.taegis.secureworks.com
AP (Sydney)	api.hotel.taegis.secureworks.com

Additional regions exist (**india**, **juliet**, **kilo**, **quebec**); use the host Secureworks shows for the tenant.

Verifying the setup

Before creating the Crogl connector, confirm the credential mints a token. From a host that can reach the region API:

```
curl --basic -u "$CLIENT_ID:$CLIENT_SECRET" \
  -H "Content-Type: application/json" \
  -d '{"grant_type": "client_credentials"}' \
```

```
https://<region_host>/auth/api/v2/auth/token
```

- **200 with an `access_token`** — the credentials are valid; proceed to create the connector in Crogl.
- **401 / 403 on the token call** — the `client_id/client_secret` pair is wrong or the client is disabled (a bad **credential**), distinct from a token that mints fine but then gets **403 on a GraphQL query**, which means the credential is right but the client lacks the read role for that resource.
- **A token that mints here but fails at connector-create in Crogl** — likely the token-exchange style. This guide's `curl` uses HTTP Basic auth with a JSON body; the connector defaults to sending the credentials in a JSON body (`client_auth_method: "json"`). If Crogl's connect-time validation fails, the connector's `client_auth_method` may need to be `"header"` instead — raise this with whoever maintains the connector bundle.

After creating the connector instance, verify end-to-end with a bounded query (for example, alerts over the last day) and confirm rows return.

Reference documentation

Every claim in this guide and in the connector's `SKILL.md` (the token endpoint and exchange style, region hosts, GraphQL operations/inputs/selection sets, and pagination shapes) was checked against these Secureworks Taegis sources directly, not recalled from training data. The GraphQL bundle was validated offline against the code-generated Python SDK because a live tenant was not available:

- [Taegis XDR API documentation](#) — the GraphQL gateway (`POST {region_host}/graphql`), client-credentials auth model, and the per-region API host list templated into `region_host`.
- [Taegis API, Getting Started / Authentication](#) — the `/auth/api/v2/auth/token` client-credentials token endpoint and the JSON-body `{"grant_type": "client_credentials"}` exchange this guide's `curl` and the connector's `client_auth_method: "json"` are based on.
- [secureworks/taegis-sdk-python](#) — the code-generated schema (services `alerts`, `investigations2`, `assets2` in `queries.py/types.py`) used to verify every GraphQL operation, input type, variable name, selection-set field, and pagination mode in this connector, since no live tenant was available to probe.
- [Taegis Query Language \(CQL\)](#) — the CQL `WHERE`-clause syntax passed to the `alerts` and `investigations` search operations.